My_sniffer

Analyseur de paquets reseau ecrit de zero — fiche de comprehension

1. Ce que demande le sujet

Ecrire un analyseur de paquets reseau, en C ou C++, **sans aucune bibliotheque reseau**. La capture, la lecture et le decodage des paquets doivent etre ecrits entierement a la main. Le projet doit fonctionner sur Linux et sur au moins un autre systeme, ne produire aucun warning, et ne jamais planter.

Le sujet insiste sur un point: ce n'est pas un exercice, c'est un produit. La compilation, l'installation, la documentation et le versionnage comptent autant que le code.

Etape	Ce qu'elle demande
0	Des fonctions maison pour lire les nombres et les adresses en binaire
1	Capturer les paquets et decoder Ethernet, IPv4, TCP et UDP
2	Filtrer les paquets selon des regles qu'on definit soi-meme
3	Reconnaitre les comportements anormaux
4	Compilation, installation, documentation, versionnage

2. Un paquet reseau, concretement

Un paquet est une suite d'octets qui commence par plusieurs en-tetes empiles, puis les donnees. Chaque en-tete repond a une question differente.

En-tete	Ce qu'il dit	Taille
Ethernet	de quelle carte reseau a quelle carte, sur le reseau local	14 octets
IPv4	de quelle machine a quelle machine, a travers Internet	20 octets au minimum
TCP ou UDP	pour quel programme, via un numero de port	20 ou 8 octets

Pourquoi trois couches et pas une seule: parce que chacune peut changer sans toucher aux autres. Un paquet qui traverse du Wi-Fi puis de la fibre voit son en-tete Ethernet reecrit a chaque saut, alors que son en-tete IP reste identique d'un bout a l'autre du trajet.

Une ligne de sortie du programme se lit ainsi:

```
02:03:21.181083  c8:5e:a9:53:40:3a > b4:7a:f1:86:73:64  172.30.0.86:58760 >
89.27.238.3:443  TCP  ACK  0 octets
     heure            carte source      carte destination     machine source    machine
dest.   proto  drapeaux  donnees
```

3. Comment le programme fonctionne

Chaque paquet traverse cinq etapes, et chaque etape correspond a un dossier du code.

Etape	Dossier	Ce qui s'y passe
1. Capturer	`src/capture/`	Demander les octets bruts au systeme. Seule etape qui demande des privileges.
2. Decoder	`src/dissect/`	Lire les en-tetes, un fichier par protocole.
3. Filtrer	`src/filter/`	Decider quels paquets garder, selon une regle.
4. Analyser	`src/analysis/`	Compter, puis reperer les comportements anormaux.
5. Rendre	`src/cli/` , `src/storage/`	Afficher, ou enregistrer dans un fichier `.pcap` .

Deux dossiers servent tout le monde: `src/net/` contient les outils de base (lecture d'octets, adresses, sommes de controle) et `src/log/` tient le journal d'execution.

Les dependances vont dans un seul sens: la ligne de commande connait tout le monde, les dossiers du milieu ne connaissent que `net` , et `net` ne connait personne. C'est ce qui permet de tester le decodage sans capturer quoi que ce soit.

4. Comment on capture, sans bibliotheque

Par defaut, une carte reseau jette les trames qui ne lui sont pas destinees. On demande au systeme de ne plus les jeter et de nous les donner brutes.

```
socket(AF_PACKET, SOCK_RAW, ETH_P_ALL)   // Linux
open("/dev/bpf") + ioctl(BIOCSETIF, ...)  // FreeBSD et macOS
```

Les deux systemes n'offrent pas du tout la meme interface: Linux rend un paquet par appel, la famille BSD en rend plusieurs d'un coup dans un tampon. D'ou deux fichiers separes, dont un seul est compile selon la machine.

> **La contrainte du sujet, prise au serieux**
>
> La fonction `htons`, qui convertit un nombre en ordre reseau, vient de `<arpa/inet.h>`, un en-tete reseau. Le projet la reecrit donc en six lignes. De meme, les entiers des en-tetes sont relus octet par octet plutot qu'avec un cast: cela evite aussi les problemes d'alignement et d'ordre des octets.

5. Comment on decode

Chaque protocole a un format fixe decrit par sa norme. Ethernet: six octets pour le destinataire, six pour la source, deux pour le type. Le champ type indique quel decodeur appeler ensuite.

Certains en-tetes portent leur propre taille. Dans IPv4, les quatre bits de poids faible du premier octet donnent la longueur de l'en-tete, comptee en mots de quatre octets.

6. Le point de securite, le coeur du sujet

Un analyseur lit par definition des donnees ecrites par quelqu'un d'autre, potentiellement hostile. **Chaque champ de longueur dans un paquet est un mensonge possible.** Le sujet sanctionne d'ailleurs tout plantage par la note maximale de la partie concernee.

Le piege classique, en trois caracteres:

```
if (offset + count <= size)   // FAUX
if (count <= size - offset)   // correct
```

Avec un `offset` enorme, la premiere version deborde: la somme repasse a zero et la condition devient vraie. Une lecture hors limites passe alors pour valide. C'est une faille exploitable.

> **Comment on prouve qu'il n'y a pas de plantage**
>
> Chaque paquet de test est coupe a
> **toutes les positions possibles**
> , une par une, et le programme doit repondre a chaque fois. Un paquet de 54 octets donne donc 55 tests a lui seul. Le tout recompile sous AddressSanitizer et verifie par Valgrind, qui rapportent zero erreur et zero fuite.

7. Comment on prouve que le decodage dit vrai

Ne pas planter ne suffit pas: encore faut-il lire juste. Le programme enregistre donc ses captures au format `.pcap`, celui de tcpdump et de Wireshark, et un script fait relire le meme fichier par ces outils.

Ils ne partagent aucune ligne de code avec le notre. S'ils extraient les memes heures, les memes adresses et les memes ports des memes octets, la lecture est juste. Verifie sur une capture reelle de cinquante paquets et sur huit fichiers du projet tcpdump, dont plusieurs volontairement casses: aucune difference.

Cette confrontation a d'ailleurs trouve trois vrais defauts qu'une relecture du code n'aurait pas vus: une etiquette VLAN empilee mal sautee, deux drapeaux TCP manquants, et une longueur incoherente rognee en silence.

8. Le langage de filtrage

Le travail se fait en deux temps, comme dans tout analyseur de texte. On decoupe la regle en mots, puis on la lit de gauche a droite en construisant un arbre, avec une fonction par niveau de priorite.

```
tcp and port 80          devient        ET
                                       /    \
                                     tcp   port 80
```

Pour savoir si on garde un paquet, on parcourt l'arbre en posant chaque question. Les parentheses et les priorites deviennent gratuites. Les noeuds vivent dans un tableau et se designent par leur position plutot que par un pointeur: aucune allocation a la main, donc aucune fuite possible.

```
mysniffer read -r capture.pcap -f "tcp and dst port 443"
mysniffer read -r capture.pcap -f "tcp-flags syn and not tcp-flags ack"
```

9. La detection des comportements anormaux

Le programme recoit les paquets un par un et remplit des compteurs, sans rien juger. Le verdict tombe a la fin, quand tout a ete vu. C'est necessaire: un pic n'a de sens que compare a une moyenne, et une regularite ne se voit qu'apres plusieurs mesures.

Comportement cherche	Comment on le detecte
Beaucoup de requetes vers un service	le **debit** de paquets, mesure sur la fenetre propre a ce service
Beaucoup d'erreurs en retour	compteur de refus (paquets TCP RST) par service
Pics de trafic	paquets par seconde compares a la moyenne
Connexions non autorisees	adresses publiques contactees, hors reseaux declares connus
Protocole inattendu sur un port standard	signature du debut des donnees sur les ports 80, 443 et 22
Intervalles reguliers	ecart moyen des intervalles a leur moyenne

Le dernier merite une explication. Un programme qui appelle son serveur de commande **dort** entre deux appels, donc il revient a heure fixe. Un humain qui navigue, jamais. On mesure donc a quel point les intervalles s'ecartent de leur moyenne: proche de zero, c'est une machine.

> **L'histoire a raconter au jury**
>
> Les detecteurs marchaient sur une capture de laboratoire. Testes sur dix minutes de navigation ordinaire, ils ont produit
> **sept alertes, toutes fausses**
> .
>
> Le detecteur de requetes comparait un nombre total: 800 paquets echanges avec un site pendant dix minutes le declenchaient, comme 800 paquets en deux secondes. Il compare maintenant un
> **debit**
> , mesure sur la fenetre propre a chaque service. Un balayage se voit, une page web non.
>
> Les balises ne comptent qu'au-dela de dix secondes d'intervalle, sinon une connexion active qui envoie un paquet par seconde paraissait parfaitement reguliere. Et l'option `--allow` declare connus les reseaux legitimes: sans elle, le programme ne peut pas parler de connexion *non autorisee*, il ne sait pas ce qui est autorise.
>
> Resultat sur le meme trafic:
> **zero fausse alerte**
> , et la capture piegee declenche toujours ses six detecteurs. Un detecteur qui crie tout le temps est un detecteur qu'on apprend a ignorer.

10. Deux surprises du monde reel

Les sommes de controle des paquets emis sont fausses. Sur du materiel moderne, la carte reseau les calcule elle-meme, juste avant de mettre la trame sur le cable, donc apres la capture. Le programme reconnait ses propres paquets a leur adresse source et l'annote au lieu de crier a l'erreur.

On voit passer des paquets de 34 000 octets alors qu'une trame Ethernet en fait 1518 au maximum. La carte fusionne plusieurs trames recues avant de les remettre au systeme. Ces deux mecanismes s'appellent *segmentation offload* et *receive offload*.

11. Chiffres et etat

Element	Valeur
Code source	environ 5 000 lignes, 9 modules
Tests	528 verifications, aucune bibliotheque de test
Dependances externes	aucune
Warnings a la compilation	zero, avec `-Wall -Wextra -Werror`
Valgrind	zero fuite, zero erreur, sur tous les chemins
Sanitizers	zero erreur memoire
Compilateurs valides	GCC 16 et GCC 14
Filtre compare a tcpdump	identique sur 13 regles differentes

Ce qu'il faut assumer devant le jury

Le code BSD et macOS n'a jamais tourne. Il est ecrit et documente, mais il n'a ete ni compile ni execute nulle part: il n'existe ni Mac ni machine FreeBSD sous la main. L'integration continue est le seul moyen de le prouver, et elle n'a pas encore tourne.

Les scripts d'installation par systeme manquent. Le sujet les demande explicitement, avec la gestion des privileges de capture. Ils ne sont pas ecrits.

Dire ces deux choses avant qu'on vous les demande vaut mieux que de les laisser decouvrir. Un manque annonce est une limite; un manque cache est un mensonge.

13. Comment on a fait, dans le detail

Cette section repond aux questions du type *« montre-moi comment tu lis cet en-tete, et pourquoi comme ca »*. Chaque reponse cite la norme qui la justifie; la liste complete des references est en section 14.

Lire l'en-tete Ethernet

Il fait toujours 14 octets: 6 pour le destinataire, 6 pour l'expediteur, 2 pour le type de ce qui suit. Il n'y a rien a calculer, les positions sont fixes.

```
if (!has_bytes(size, 0, 14))
    return false;

out->destination = mac_from_bytes(data);
out->source      = mac_from_bytes(data + 6);
out->type        = read_u16(data, 12);
out->payload_offset = 14;
```

Pourquoi la verification d'abord. On recoit une taille en meme temps que les octets. Si la trame a ete coupee, lire le 14e octet sortirait de la memoire du paquet. C'est la regle appliquee partout dans le projet.

Source: IEEE 802.3 pour le format de la trame, RFC 894 pour le transport d'IP dessus.

Savoir ce qu'il y a en dessous

Le champ des octets 12 et 13 a deux sens possibles. S'il vaut 1536 ou plus, c'est un numero de protocole. En dessous de 1500, c'est une longueur, et on a affaire a un ancien format. Cette frontiere n'est pas une convention maison: elle est fixee par IEEE 802.3.

Valeur	Ce que ca veut dire
`0x0800`	IPv4 suit
`0x0806`	ARP suit
`0x86DD`	IPv6 suit
`0x8100`, `0x88A8`	une etiquette VLAN de 4 octets s'intercale

Les etiquettes VLAN peuvent s'empiler. On boucle donc, **avec une limite de deux**: c'est ce que permet la norme, et une boucle sans borne serait une invitation a nous faire tourner indefiniment sur un paquet fabrique expres.

Source: IEEE 802.1Q pour les etiquettes, registre IANA des EtherTypes pour les valeurs.

Lire un nombre de deux octets

```
uint16_t read_u16(const uint8_t *data, size_t offset)
{
    return (data[offset] << 8) | data[offset + 1];
}
```

Sur le reseau, l'octet de poids fort vient toujours en premier. Les processeurs Intel rangent l'inverse en memoire. On recompose donc le nombre a la main plutot que de copier les octets dans une variable.

Trois raisons de ne pas faire un cast vers une structure: l'alignement n'est pas garanti, l'ordre des octets de la machine n'est pas celui du reseau, et le compilateur peut inserer du remplissage entre les champs. La lecture manuelle evite les trois d'un coup.

Source: RFC 791, annexe B, qui pose l'ordre de transmission.

Trouver la taille de l'en-tete IPv4

Contrairement a Ethernet, il n'a pas de taille fixe: il peut porter des options. Sa taille est ecrite dedans, dans les 4 bits de poids faible du premier octet, comptee en mots de 4 octets.

```
uint8_t version_and_length = read_u8(data, offset);

out->version = version_and_length >> 4;      // 4 bits de poids fort
if (out->version != 4)
    return false;

size_t words = version_and_length & 0x0f;    // 4 bits de poids faible
if (words < 5)                               // 5 mots = 20 octets, le minimum legal
    return false;
out->header_length = words * 4;
```

Pourquoi refuser moins de 5. Un en-tete IPv4 fait au minimum 20 octets. Une valeur plus petite ne peut pas exister sur un vrai reseau: c'est soit un paquet abime, soit un paquet fabrique pour faire lire le programme en arriere.

Source: RFC 791, section 3.1.

Verifier une somme de controle

On additionne toute la zone deux octets par deux octets, on replie les retenues qui depassent 16 bits, et on inverse tous les bits. Pour **verifier** un en-tete recu, on refait le calcul sur l'en-tete complet, champ de somme inclus: si rien n'a ete abime, le resultat vaut zero.

TCP et UDP compliquent un peu: leur somme couvre aussi un **pseudo en-tete** fabrique a partir des deux adresses IP. Ce n'est pas envoye sur le reseau, c'est juste ajoute au calcul. Son role est de detecter un paquet livre au mauvais destinataire.

Source: RFC 1071 pour l'algorithme, RFC 9293 et RFC 768 pour les pseudo en-tetes.

Les drapeaux TCP

Ils tiennent dans un octet, un bit chacun. Un detail compte: le bit 7 s'appelait **NS** dans un mecanisme abandonne depuis. L'IANA l'a reattribue en 2026 sous le nom **AE**. Beaucoup de tutoriels affichent encore NS.

Source: RFC 9293, qui remplace la RFC 793; RFC 3168 pour ECE et CWR; RFC 9768 pour AE.

Ouvrir la capture

```
socket(AF_PACKET, SOCK_RAW, to_network_order(ETH_P_ALL));
```

`SOCK_RAW` demande les trames avec leur en-tete Ethernet; `SOCK_DGRAM` nous le retirerait. `ETH_P_ALL` veut dire tous les protocoles. Puis un `bind` sur une seule interface, sinon on recevrait des en-tetes de formats differents melanges.

Le detail a raconter. Le mode espion est demande en faisant adherer la socket a un groupe, et non en modifiant la carte reseau. L'adhesion demande un privilege plus faible, et surtout le noyau l'annule tout seul si le programme meurt. En modifiant la carte, elle resterait en mode espion apres un plantage, sans que personne s'en apercoive.

Source: page de manuel `packet(7)` sous Linux, `bpf(4)` sur FreeBSD et macOS.

Ecrire un fichier .pcap

Un en-tete de 24 octets au debut du fichier, puis pour chaque paquet un en-tete de 16 octets suivi de ses octets bruts. Le fichier commence par un nombre magique qui dit a la fois « ceci est un pcap » et dans quel ordre d'octets il a ete ecrit.

A la lecture, on decode ces quatre premiers octets **dans les deux sens** et on regarde lequel donne une valeur connue. Cela rend le lecteur insensible a la machine qui a produit le fichier.

Source: le format n'est pas un RFC, c'est un brouillon IETF au statut historique. La reference qui fait foi est l'implementation de libpcap et la page `pcap-savefile(5)`.

14. Les sources

Chaque decision technique du projet s'appuie sur un document. Les dossiers complets, avec citations et liens, sont dans `docs/defense/research/`.

Les protocoles

Sujet	Reference
Trame Ethernet, seuil 1536	IEEE 802.3
IP sur Ethernet, type 0x0800	RFC 894
Etiquettes VLAN	IEEE 802.1Q
Valeurs des EtherTypes	registre IANA *IEEE 802 Numbers*
En-tete IPv4	RFC 791, section 3.1
Ordre des octets sur le reseau	RFC 791, annexe B
Somme de controle Internet	RFC 1071
En-tete TCP	RFC 9293 (remplace la RFC 793)
Drapeaux ECE et CWR	RFC 3168
Drapeau AE, ex-NS	RFC 9768
En-tete UDP	RFC 768
Numeros de protocole, numeros de port	registres IANA
Adresses privees	RFC 1918, et le registre IANA des adresses a usage special

Les systemes

Sujet	Reference
Capture sous Linux	page de manuel `packet(7)`
Capture sous FreeBSD et macOS	page de manuel `bpf(4)`
Liste des interfaces	`getifaddrs(3)`
Privileges de capture	`capabilities(7)`, `setcap(8)`
Abandon des privileges	Chen, Wagner et Dean, *Setuid Demystified*, USENIX 2002
Format de fichier pcap	`pcap-savefile(5)` et le code de libpcap

La detection d'anomalies

Detecteur	Reference
Balayage de ports	Jung, Paxson, Berger et Balakrishnan, IEEE S&P 2004
Pics de trafic	Brutlag, LISA 2000; moyennes mobiles du NIST
Protocole sur un port inattendu	Dreger et al., USENIX Security 2006; MITRE ATT&CK T1571
Appels reguliers (balises)	methode RITA; MITRE ATT&CK T1071
Adresses non routables	RFC 1918, RFC 6890

> **Si on vous demande une reference que vous ne retrouvez pas**
>
> Repondez ce que vous savez: *« l'en-tete Ethernet fait 14 octets, c'est IEEE 802.3, et je l'ai verifie en comparant ma sortie a celle de tcpdump sur les memes octets »*. Un numero de RFC oublie se pardonne. Une reponse inventee, non.

15. Les trois phrases a retenir

J'ouvre une socket brute pour recevoir les octets de la carte reseau, et je decode les en-tetes a la main, sans aucune bibliotheque reseau.

Avant chaque lecture je verifie qu'il reste assez d'octets, parce qu'un paquet fabrique expres ferait sinon planter le programme, et le sujet le sanctionne.

Je prouve que mon decodage est juste en enregistrant un fichier et en le faisant relire par tcpdump et Wireshark: ils lisent les memes heures, les memes adresses et les memes ports que moi.